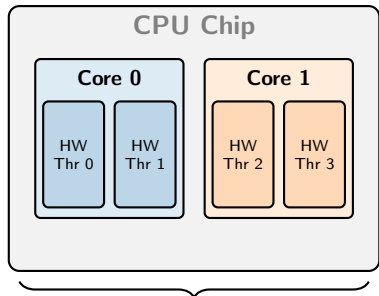


Concurrency in Python: Threads and Processes

IO-Bound vs. CPU-Bound Work and the GIL

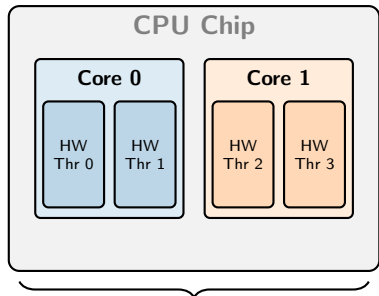
Cores, Hardware Threads, and Software Threads



$2 \text{ cores} \times 2 \text{ HW threads} = 4 \text{ logical processors}$

CPU Core — a physical processing unit on the chip. Each core executes instructions independently.

Cores, Hardware Threads, and Software Threads

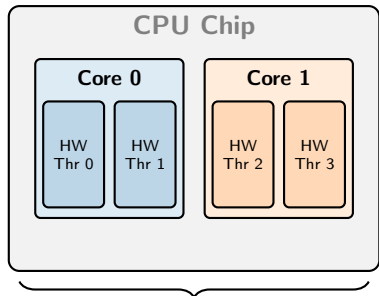


2 cores $\times$ 2 HW threads = 4 logical processors

CPU Core — a physical processing unit on the chip. Each core executes instructions independently.

Hardware Thread (SMT) — each core presents as 2+ logical processors. When one stalls on a memory fetch, the other uses the execution units.

Cores, Hardware Threads, and Software Threads



$2 \text{ cores} \times 2 \text{ HW threads} = 4 \text{ logical processors}$

CPU Core — a physical processing unit on the chip. Each core executes instructions independently.

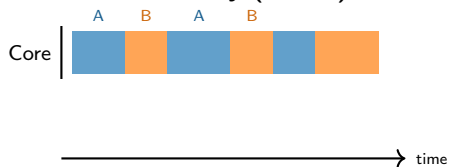
Hardware Thread (SMT) — each core presents as 2+ logical processors. When one stalls on a memory fetch, the other uses the execution units.

Software Thread — an OS-scheduled sequence of instructions. A program can create thousands, regardless of hardware thread count.

This presentation focuses on software threads.

Concurrency vs. Parallelism

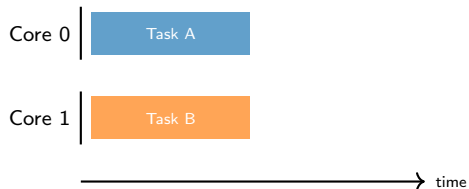
Concurrency (1 core)



Concurrency = managing multiple tasks with overlapping lifetimes.

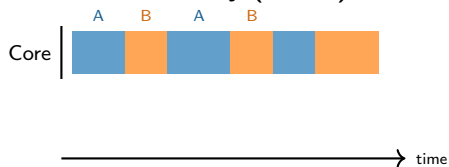
Tasks *interleave* on one core — only one runs at any instant.

Parallelism (2 cores)

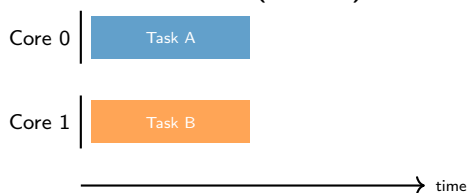


Concurrency vs. Parallelism

Concurrency (1 core)



Parallelism (2 cores)



Concurrency = managing multiple tasks with overlapping lifetimes.

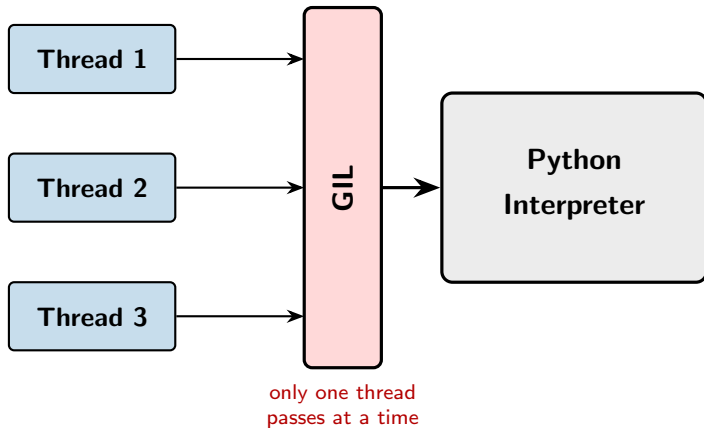
Tasks *interleave* on one core — only one runs at any instant.

Parallelism = executing tasks *simultaneously* on different cores.

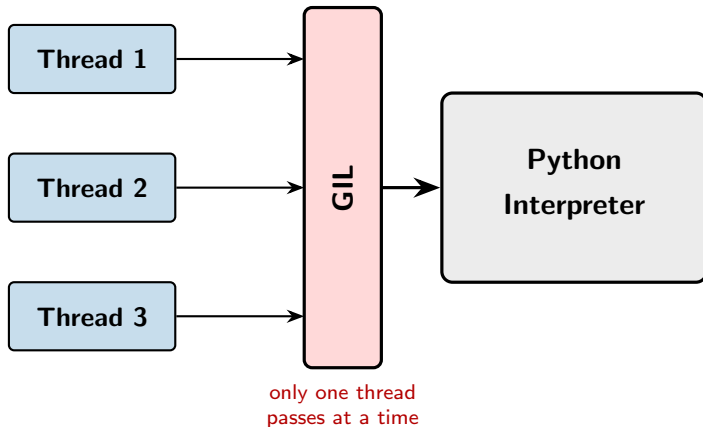
Parallelism is a *subset* of concurrency.

Key distinction: Concurrency is about *structure*, parallelism is about *execution*.

Python's Global Interpreter Lock (GIL)



Python's Global Interpreter Lock (GIL)



The GIL is **released** during IO operations (network calls, file reads, `time.sleep`).
→ Threads *can* overlap IO waits even though only one runs Python bytecode at a time.

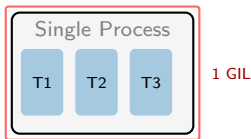
Two Approaches: Threads vs. Processes

	Threads	Processes
Mechanism	Shared memory	Separate interpreters
GIL impact	One thread runs Python at a time	Each process has its own GIL
Data sharing	Direct (same address space)	Serialized (pickled)
Startup cost	Lightweight	Heavier
Best for	IO-bound tasks	CPU-bound tasks

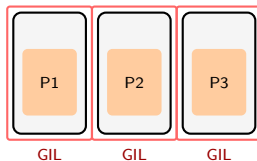
Two Approaches: Threads vs. Processes

	Threads	Processes
Mechanism	Shared memory	Separate interpreters
GIL impact	One thread runs Python at a time	Each process has its own GIL
Data sharing	Direct (same address space)	Serialized (pickled)
Startup cost	Lightweight	Heavier
Best for	IO-bound tasks	CPU-bound tasks

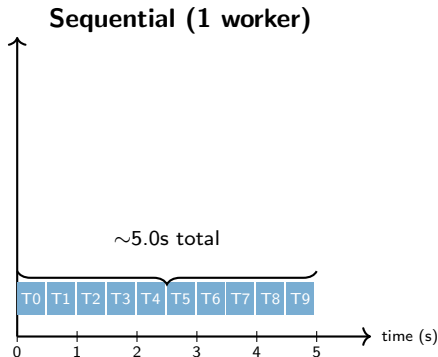
ThreadPoolExecutor



ProcessPoolExecutor



IO-Bound Tasks: Sequential Execution



A task is **IO-bound** when it spends most time *waiting* for external resources.

Examples:

- API calls
- File reads / writes
- Database queries

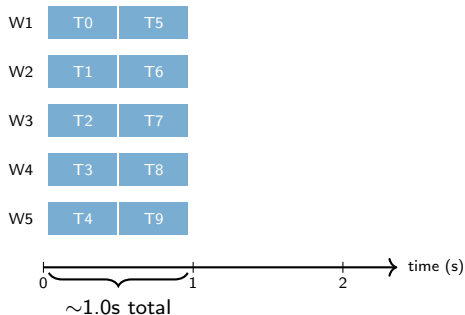
Here: 10 tasks, each taking 0.5s.

Sequential total: $10 \times 0.5 = \mathbf{5.0s}$

Each task waits for the previous one to finish.

IO-Bound Tasks: ThreadPoolExecutor

Threaded (5 workers)



```
with ThreadPoolExecutor(  
    max_workers=5,  
) as executor:  
    results = executor.map(  
        simulate_io_task,  
        range(10),  
    )
```

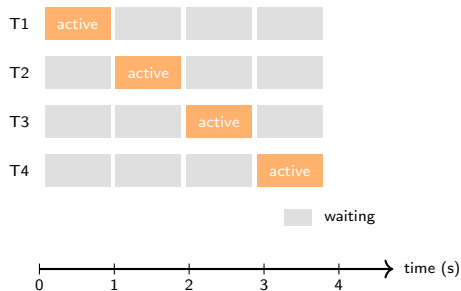
GIL is **released** during `sleep()` and real IO.

5 workers $\times$ 2 batches = $\sim 1.0s$

5 $\times$ speedup

CPU-Bound Tasks: Threads Don't Help

4 Threads (GIL serializes)



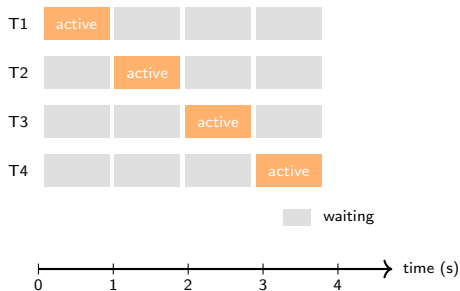
CPU-bound: task spends time *computing*, not waiting.

Method	Time
Sequential	~4s
4 Threads	~4s

Speedup: 1.0× (no improvement)

CPU-Bound Tasks: Threads Don't Help

4 Threads (GIL serializes)



CPU-bound: task spends time *computing*, not waiting.

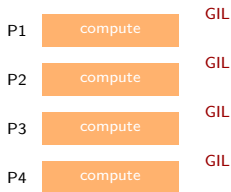
Method	Time
Sequential	~4s
4 Threads	~4s

Speedup: 1.0× (no improvement)

The GIL forces threads to take turns running Python bytecode → tasks run *sequentially* despite having 4 threads.

CPU-Bound Tasks: ProcessPoolExecutor

4 Processes (true parallelism)



```
with ProcessPoolExecutor(  
    max_workers=4,  
) as executor:  
    results = executor.map(  
        is_prime,  
        numbers,  
    )
```

Same API — just swap the class name.
Each process has **its own GIL** → true parallel execution.

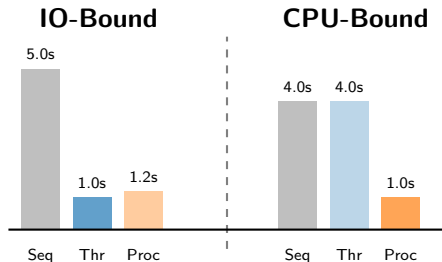
~4× speedup

Summary: Speedup Comparison

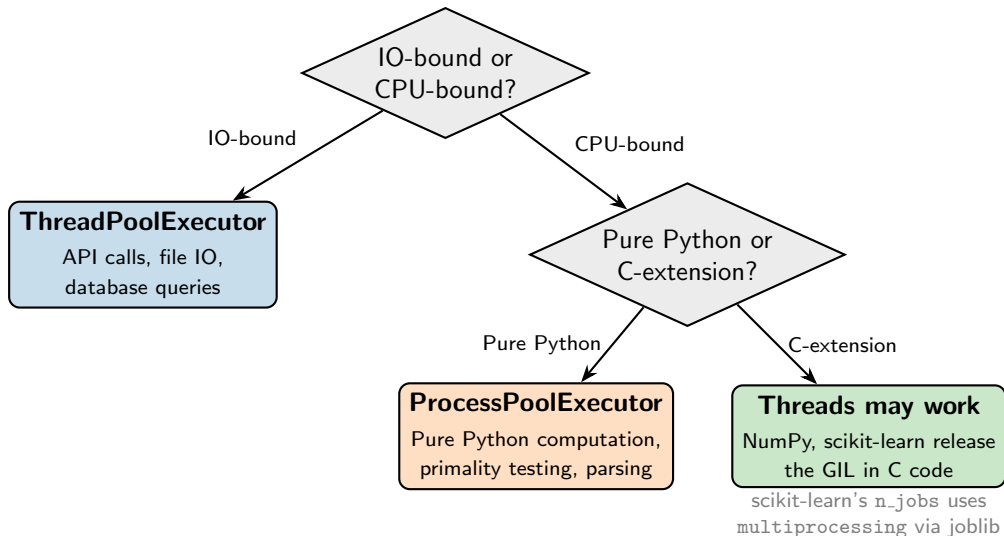
Task Type	Method	Approx. Time	Speedup
IO-Bound (10 tasks)	Sequential	5.0s	1.0×
IO-Bound	Threads (5 workers)	1.0s	5.0×
IO-Bound	Processes (5 workers)	1.2s	4.2×
CPU-Bound (8 tasks)	Sequential	4.0s	1.0×
CPU-Bound	Threads (4 workers)	4.0s	1.0×
CPU-Bound	Processes (4 workers)	1.0s	4.0×

Summary: Speedup Comparison

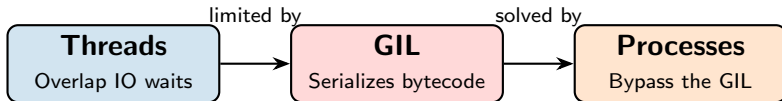
Task Type	Method	Approx. Time	Speedup
IO-Bound (10 tasks)	Sequential	5.0s	1.0×
IO-Bound	Threads (5 workers)	1.0s	5.0×
IO-Bound	Processes (5 workers)	1.2s	4.2×
CPU-Bound (8 tasks)	Sequential	4.0s	1.0×
CPU-Bound	Threads (4 workers)	4.0s	1.0×
CPU-Bound	Processes (4 workers)	1.0s	4.0×



Decision Guide: Threads or Processes?



Key Takeaways



Key Takeaways:

1. **IO-bound** tasks (API calls, file IO) → use `ThreadPoolExecutor`
2. **CPU-bound** tasks (pure Python computation) → use `ProcessPoolExecutor`
3. The GIL prevents threads from speeding up CPU-bound Python code
4. Both use the same `concurrent.futures` API — just swap the class name
5. Start sequential, profile first, add concurrency only where it helps

Connection to ML: scikit-learn's `n_jobs` parameter uses multiprocessing via `joblib`. Setting `n_jobs=-1` uses all cores — now you know the mechanism behind it.